Reinforcement Learning Level Generator

Instructions

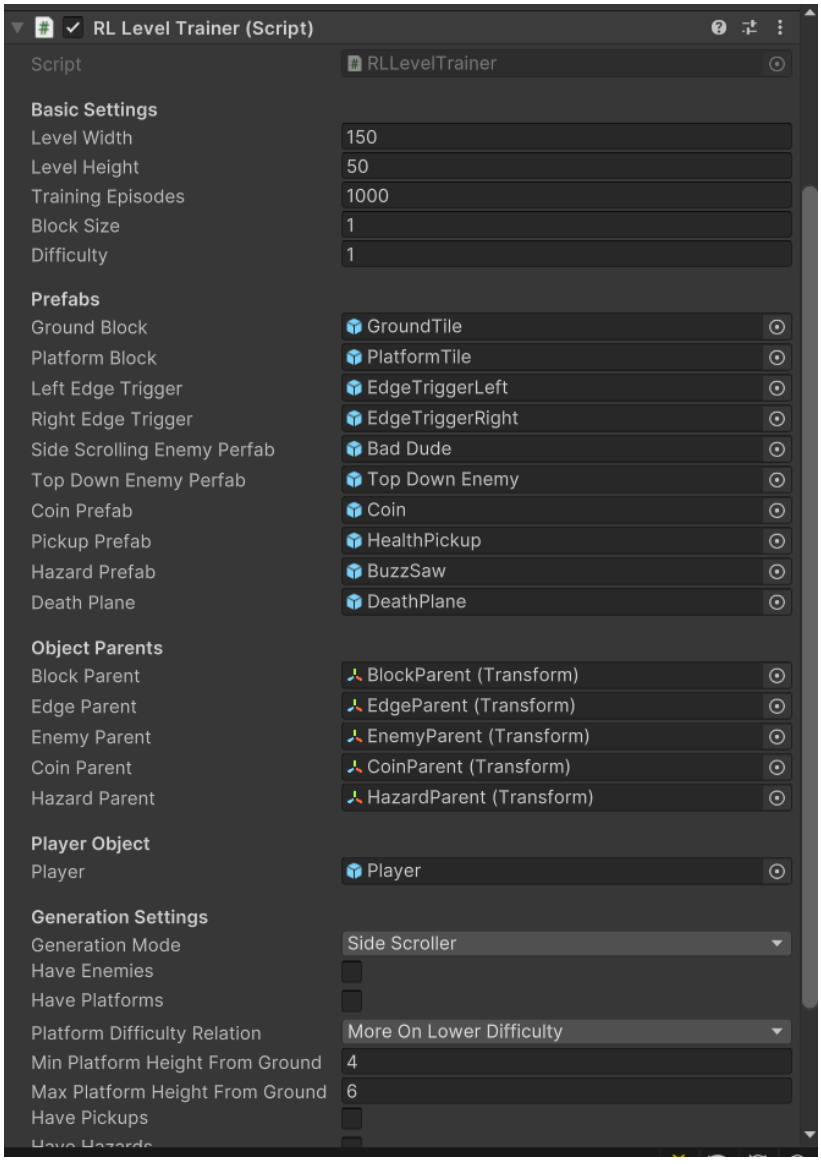
Afonso Costa

Table of Contents

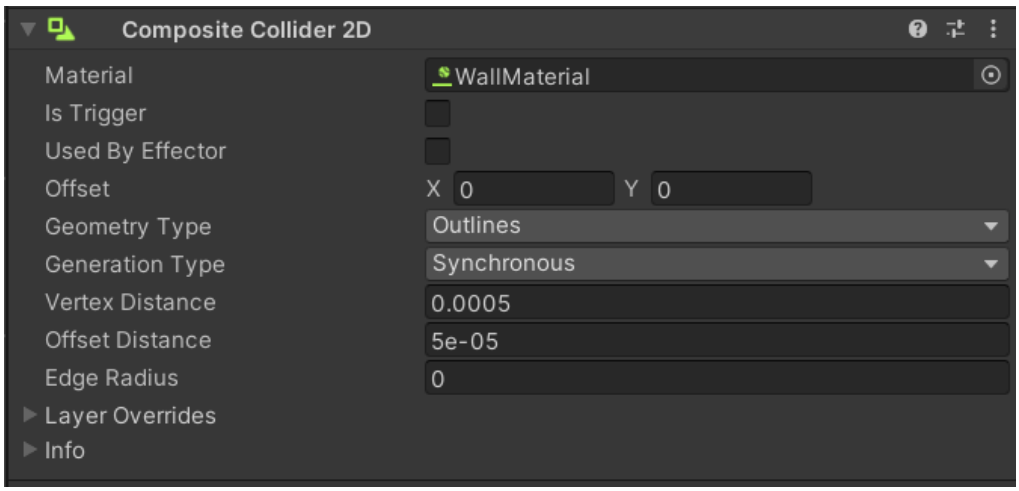
Section 1. Setup

1. Make sure the following files are present:
    - GeneratorAction.cs
    - GeneratorState.cs
    - RLGeneratorAgent.cs
    - RLLevelTrainer.cs
    - PlayerPerformanceTracker.cs
    - EnemyDifficultyAdjuster.cs
    - TileType.cs

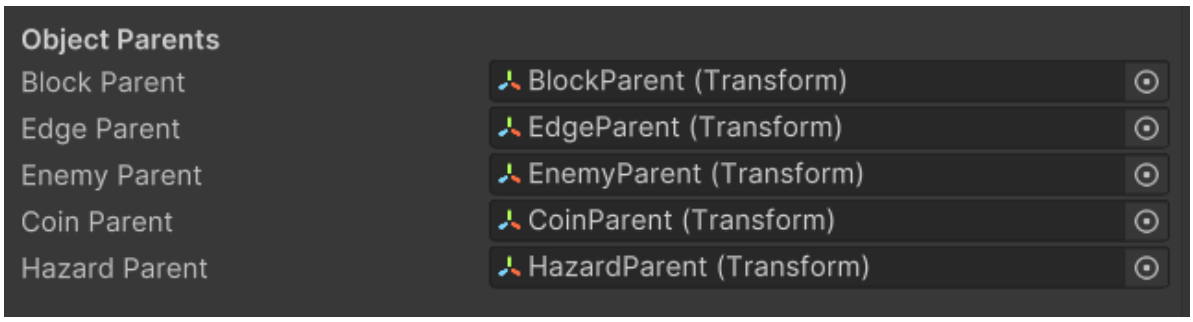
2. Create an Empty Game Object in your Unity scene (the name of the object is irrelevant) and attach the RLLevelTrainer.cs as a component. It should look like the image below. Note that the fields in the Inspector will be empty, unlike how they are shown in the image.



3. Create an Empty Game Object (the name of the object is irrelevant) and give it the Composite Collider 2D. This is so that all the colliders from the block that will be instantiated get merged into a single collider to avoid collision issues. The component should look like the image below.

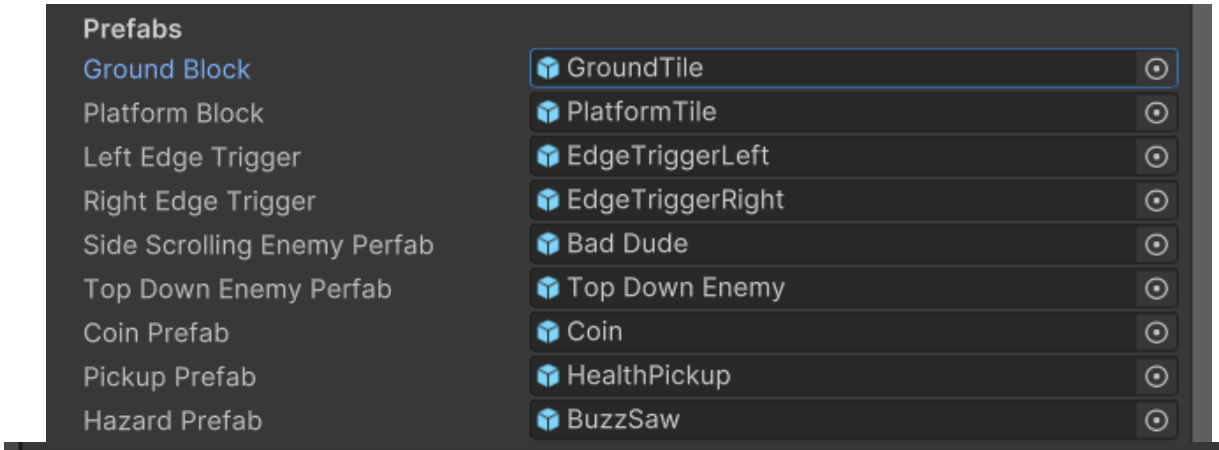


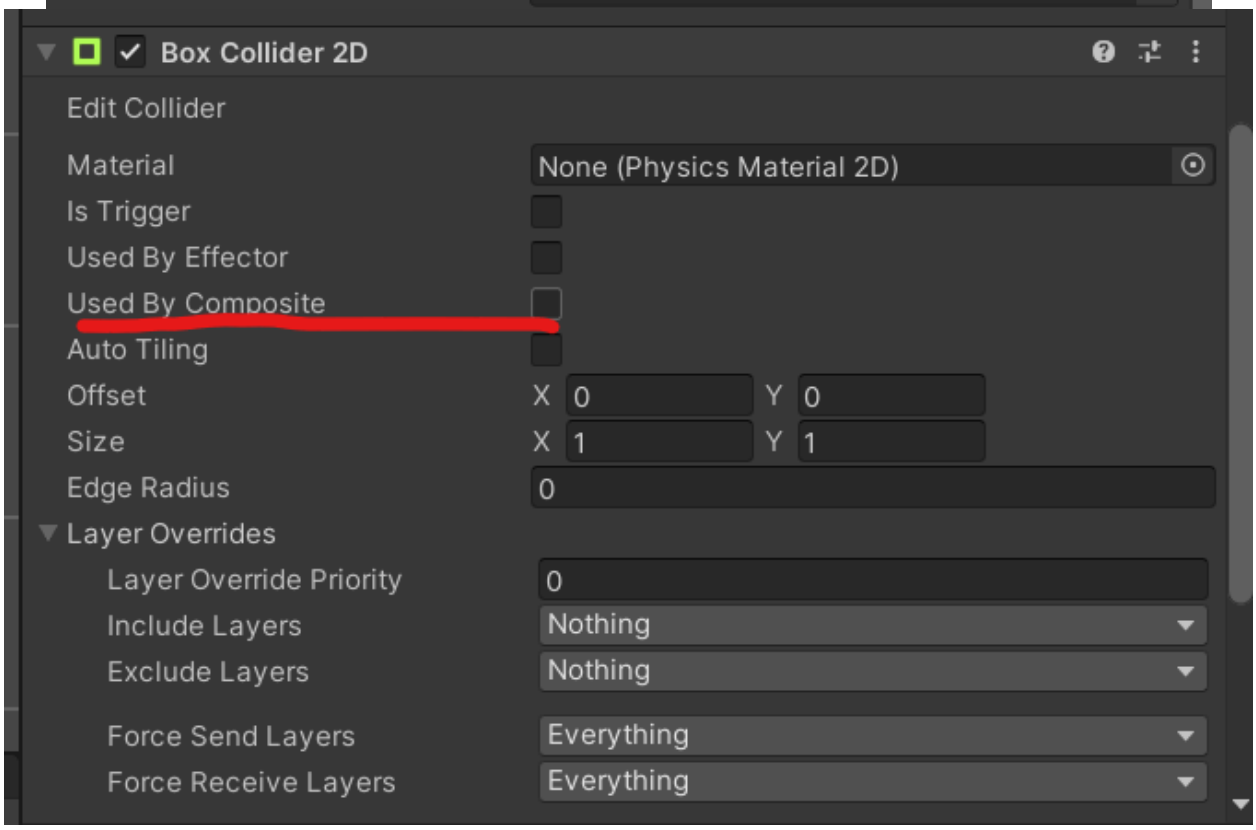
4. Place the object created in step 3 in the “Block Parent” field found in the “Object Parents” section in your Object with the RLLevelTrainer.cs script, seen in the image below. This will serve as the parent object for all the blocks that will make up the ground and platforms. This is done to keep the scene’s hierarchy organized.



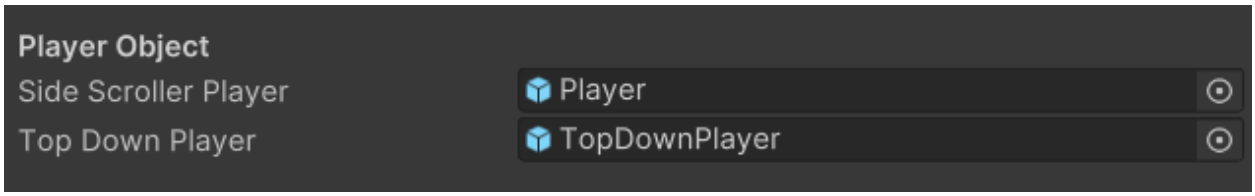
5. Create 4 Empty Game Objects (the names of the objects are irrelevant) and place them in the remaining “Parent” fields in the section mentioned in step 4. These will also serve to keep the scene’s hierarchy organized.

6. In the “Prefabs” section of the RLLevelTrainer.cs, seen in the first image below, place the prefabs you wish to be used by the generator to create the levels. Note that the prefabs used in the “Ground Block” and “Platform Block” fields need to have the Box Collider 2D component, and must have the “Used By Composite” option selected, as seen in the second image below. The “Death Plane”, “Left Edge Trigger”, and “Right Edge Trigger” should all be Trigger objects. “Death Plane” is used to detect if your player falls off the map, and the “Edge Triggers” are meant to stop enemies from falling off the map, being placed automatically after the ground and platforms are created.





7. Place your Player Object in the Scene, and place that instance in the “Player” field of the RLLevelTrainer.cs that corresponds to the type of level you want to generate (“Side Scroller Player” for Side Scrolling levels and “Top Down Player” for Top Down levels). The player’s position will be updated when the level is done generating. You don’t have to place anything in the field that doesn’t correspond to the type of level you wish to generate. If you wish to generate both types of levels, both player objects need to be present in the scene.

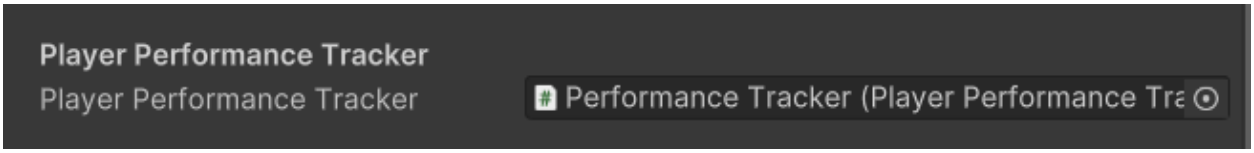


8. On the “Basic Settings” section of the RLLevelTrainer.cs, set the “Block Size” field to the size of the prefabs you intend to use for the ground and platforms. These should be cubes, and the recommended cube size is 1.

9. If your game has enemies, add the EnemyDifficultyAdjuster.cs script to the prefab as a component. This can be used to change the maximum health and movement speed of the enemy object by using the public variables “MaxHealth” and “MovementSpeed”.

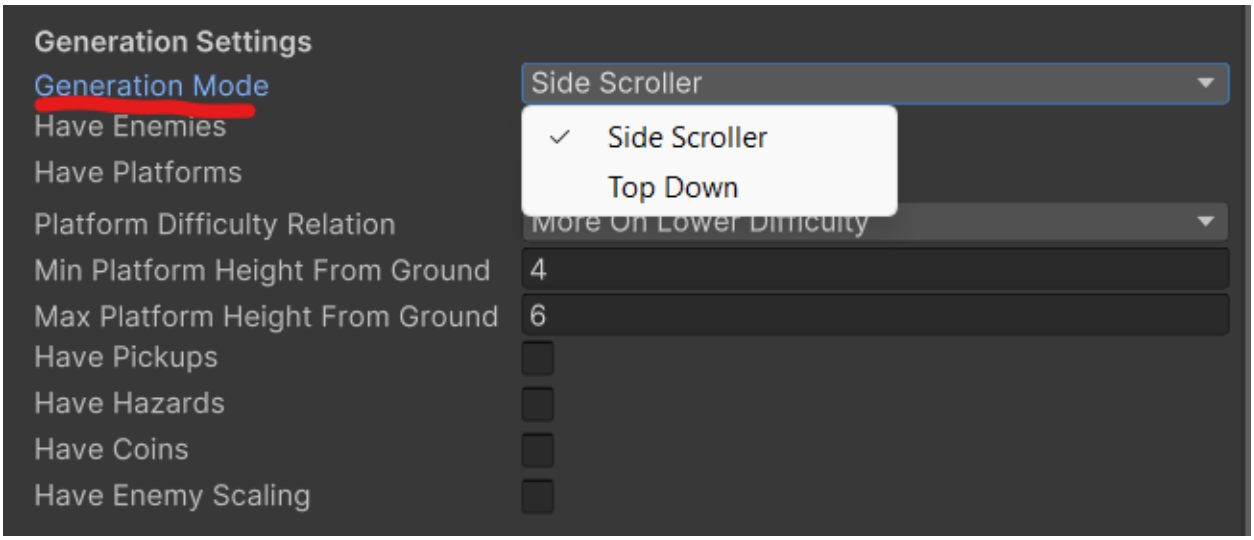
10. Create an Empty Game Object and attach the PlayerPerformanceTracker.cs script as a component. This is used to change the difficulty based on the player’s performance.

11. Place the object created in step 10 in the “Player Performance Tracker” field of the RLLevelTracker.cs, as seen in the image below.



Section 2. Utilization

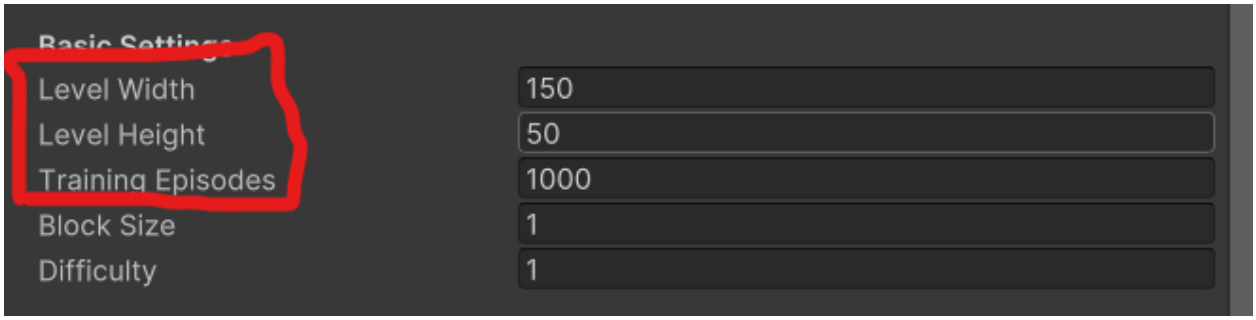
1. Select what type of level you wish to generate by clicking the “Generation Mode” option in the “Generation Settings” of the RLLevelTrainer.cs, seen in the image below. Once clicked, 2 options are available: Side Scroller and Top Down. Select one of them.



2. Depending on what Generation Mode is chosen, it is highly advised to change the values found in the “Basic Settings”, seen in the image below, section of the RLLevelTrainer.cs, mostly for the top-down levels, since they are the ones that take the longest to generate. The recommended values for each generation mode are:

    - Side Scroller:
        - Level Width: Between 100 and 200.
        - Level Height: 50 to 75.
        - Training Episodes: At least 1000.

    - Top Down:
        - Level Width: Between 20 and 40.
        - Level Height: Between 20 and 40.
        - Training Episodes: At least 1000. Avoid going over 5000.



3. Adjust the rest of the settings found in the “Generation Settings” to create the desired Level. These options include:

    - If Enemies are generated or not
    - If Platforms are generated or not
    - The minimum and the maximum height the platforms can generate from the ground (used to adjust platform generation to the player’s jump mechanics)
    - The relationship between Difficulty and the ratio of platforms that are generated. Less on lower difficulties and more on higher difficulties or vice versa.
    - If Pickup items, such as health pickups, are generated or not
    - If Stage Hazards are generated or not
    - If Collectibles, similar to coins from Super Mario or Rings from Sonic the Hedgehog, are generated or not.
    - If Enemies' behavior adapts to the Difficulty. The higher the difficulty is, the faster they move and the more health they have.

4. If the number of enemies and/or collectibles present is important for your game, the public functions “GetEnemyCount” and “GetCoinCount” return an integer value of the number of enemies and collectibles, respectively.

5. To have the difficulty adapt to the player’s performance, call the “UpdatePerformance” function, found in the PlayerPerformanceTracker.cs script, in

the functions or systems that detect if the player completed or failed the level. The “UpdatePerformance” function also calls the “AdjustDifficulty” function, which increases or decreases difficulty based on the number of “ConcecutivesWins” or “ConvecutiveLosses”. You can also change how much the difficulty is altered on victory or defeat by altering the Difficulty Increment field on the PlayerPerformanceTracker.cs.

6. To generate another level, reload your scene. The object holding your PlayerPerformanceTracker.cs will not be destroyed upon loading the scene.

Section 3. Functional Example

Functional Example Repository:
https://github.com/ItsYaBoiBirdMan/RL-2D-Level-Generator

The link above leads to a GitHub repository that contains a functional example of the RL Level Generator.

This example features a scene (MainScene) that showcases how the asset should work and the type of levels that it is able to generate, including all the necessary prefabs, such as ground blocks, platform blocks, enemies, ect., all being found in the “Prefabs” folder. It also has 2 player objects, one for the side scrolling levels and another of the top down levels. The controls for each player objects are:

- **Side Scroller Player:**
    - A - Move Left
    - D - Move Right
    - Space Bar - Jump. Press again while in the air for a double jump.
    - O - Attack. The direction of the attack will be the same as the last direction you moved in.
- **Top Down Player:**
    - W - Move North
    - A - Move West
    - S - Move South
    - D - Move East
    - O - Attack. The attack is an area of effect with the player at its center.

The scene also features a fully functional User Interface (UI), complete with a health bar for the player, counters for remaining enemies and coins, the screen fading to black when falling off the map, dying or completing the level. It also has a menu that appears when the player dies or completes the level. This menu has 2 buttons, Respawn and Quit. Respawn Generates a new Level, and Quit would close the application, but it has no function while in the Unity Editor.

If there are any doubts about how to operate the example, refer to Sections 1 and 2, as the process of setting up and using are the same as for someone using the asset in a new, empty project.